

AOI-Vision 软著申请 — 技术方案

软件名称: AOI-Vision (AOI-Vision) 基于OasysCoreCV的自动光学检测系统
软件 简称: AOI-Vision / AOI-Vision 版本: V0.2.0 著作权人: 赵达伟 / 潤芯國際
(香港)有限公司 / 昆山山之旭电子科技有限公司

一、软件基本信息

项目	内容
软件名称 (中文)	AOI-Vision (AOI-Vision) 基于 OasysCoreCV的自动光学检测系统软件
软件名称 (英文)	AOI-Vision Automated Optical Inspection System Software based on OasysCoreCV
简称	AOI-Vision / AOI-Vision
版本号	V0.2.0
开发语言	Python 3.11, HTML/CSS, JavaScript
代码行数	约 4,634 行 (不含第三方库)
运行环境	macOS 14+ / Ubuntu 22.04+ / Windows Server 2022 (Docker)
硬件环境	x86_64 CPU, 4GB+ RAM, USB UVC Camera / HaaS506-HD3 4G 模块

项目	内容
数据库	SQLite（开发）/ PostgreSQL（生产）

二、技术领域

本发明属于工业自动化与机器视觉领域，具体涉及一种基于模块化视觉引擎的自动光学检测系统，用于半导体、电子制造、精密机械等行业的在线产品质量检测、缺陷识别、精密量测和统计过程控制。

三、解决的技术问题

问题1：检测算法不可按需配置

现有 AOI 系统通常将所有检测算法打包部署，无法根据客户产线需求灵活选择启用哪些检测模块，导致客户为不需要的功能付费，同时增加系统复杂度。

问题2：核心算法缺乏商业保护

传统视觉检测系统将算法代码直接部署在业务服务器上，源代码以明文形式存在，容易被逆向工程或代码泄露，核心知识产权得不到有效保护。

问题3：权限管理过于简单

多数工业检测系统仅使用简单的管理员/普通用户二元权限模型，无法满足多角色、多权限组的精细化访问控制需求（如质检员只读检测结果、操作员只能执行检测、管理员可管理全部）。

问题4：检测模式单一

现有系统通常只支持单一种类检测（如仅模板比对），无法覆盖频域分析、特征匹配、立体视觉等多种检测场景，需要多套系统配合才能完成复杂检测任务。

问题5：跨数据库部署困难

传统系统通常与单一数据库类型深度绑定，从开发环境（SQLite）切换到生产环境（PostgreSQL）需要大量代码修改。

四、采用的技术手段

技术手段1：模块化视觉引擎注册表

描述：设计 ModuleRegistry 单例模式注册表，将 17 种检测模块以数据驱动方式定义（模块名、分类、层级、依赖关系、参数），SysAdmin 可通过 API 运行时启用/禁用任意模块，无需重启服务。

实现要点： - 每个模块定义为 EngineModule 数据类 (key/name/category/description/requires/level/enabled) - 依赖检查机制：启用模块时自动检查前置依赖是否满足 - 配置快照导出：支持将当前模块配置导出为 JSON，用于客户打包

技术手段2：Core Engine 独立包 + Cython 编译保护

描述：将所有视觉算法代码独立为顶层 aoi_engine 包，通过 sys.path 动态导入，与业务层 (FastAPI) 和 UI 层零耦合。发布时使用 Cython 将 .py 编译为 .so 二进制文件，删除源代码。

实现要点： - Core Engine 不含任何 FastAPI/数据库依赖 - setup.py 配置 cythonize 编译目标 - 业务层通过懒导入 (函数体内 import) 避免启动时依赖 - 编译后 .so 文件不可逆，无法反编译还原为源代码

技术手段3：全多对多 RBAC 权限系统

描述：实现 Permission (权限组) [?] AccountGroup (账号组) [?] User (用户) 三层多对多模型，通过 require_permission(resource, action) 装饰器实现细粒度访问控制。

实现要点： - 两张 M2M 关联表：user_account_groups + account_group_permissions - JWT payload 包含所有用户有效权限的字典 - 中间件遍历用户所属全部账号组，聚合所有关联权限组的权限 - 支持 manage/write/read 三级操作粒度 - Admin 用户持有全局权限 (短路检查)

技术手段4：多模式检测引擎

描述：DefectEngine 统一入口 `run(frame, rule, golden_frame)` → 根据规则类型路由到 8 种检测方法：模板比对、特征匹配(SIFT/ORB)、轮廓分析、颜色阈值(HSV)、频域分析(FFT)、形态学顶帽、亚像素边缘定位、目标计数。每种方法返回统一的 `InspectionResult` 数据结构。

实现要点： - 统一数据类： `Defect(type/severity/x/y/w/h/area/description)` + `InspectionResult(defect_count/pass_fail/defects/confidence)` - 导入保护：
`aoi_engine` 不可用时自动使用内置 `fallback` 引擎 - 每种检测方法独立封装，可单独测试和优化

技术手段5：跨平台数据库适配

描述：通过 SQLAlchemy 的 `TypeDecorator` 自定义 `UUID` 类型，实现 `UUID` 在 SQLite（`CHAR(36)` 字符串）和 PostgreSQL（原生 `UUID`）之间的自动转换。`DATABASE_URL` 通过 `.env` 环境变量配置，一行切换数据库类型。

实现要点： - `UUID` 类型根据 `dialect.name` 判断当前数据库类型 -
`process_bind_param: UUID → str (SQLite) / UUID (PostgreSQL)` -
`process_result_value: str/bytes → UUID` 对象统一返回 - 模型层统一使用 `UUID` 类型，不区分数据库

五、技术效果

效果1：检测模块按需配置（对应问题1）

SysAdmin 可在管理面板一键开关 17 种检测模块，客户按需付费。启用/禁用 1 秒内生效，无需重启服务。配置可导出为 JSON 快照，用于多客户快速部署。

效果2：核心算法有效保护（对应问题2）

Core Engine 编译为 .so 后，即使服务器被完全攻破，攻击者也只能获得二进制机器码，无法还原原始算法逻辑。源代码删除后，逆向成本接近重写。

效果3：精细化权限管理（对应问题3）

支持 4 个预置权限组 × 4 个预置账号组的多对多组合，可实现任意粒度的权限分配。每个 API 端点独立检查权限，拒绝无权限访问。

效果4：多模式检测覆盖（对应问题4）

单一系统集成 17 种检测模式，覆盖从基础模板比对到企业级频域分析、立体视觉的全场景检测需求。统一数据结构确保检测结果在报表和趋势分析中一致可用。

效果5：一键数据库切换（对应问题5）

修改 .env 中一行 DATABASE_URL 即可在 SQLite 和 PostgreSQL 之间切换，模型层零修改。SQLite 开发环境零依赖，PostgreSQL 生产环境高可用。

六、技术创新点总结

序号	创新点	技术特征
1	模块化视觉引擎注册表	17 模块数据驱动定义，运行时开关，依赖检查
2	Core Engine Cython 编译保护	独立包 + 懒导入 + .so 编译，源码删除
3	全多对多 RBAC	Permission↔AccountGroup↔User，三级操作粒度
4	多模式统一检测引擎	8 种检测方法，统一数据结构，统一入口
5	跨平台 UUID 适配	TypeDecorator GUID，SQLite CHAR(36) ↔ PostgreSQL UUID
6	懒加载硬件抽象层	工厂模式 + 注册表，无硬件依赖仍可 import

七、源程序量

模块	文件数	代码行数	说明
Core Engine (aoi_engine/)	7	469	defect.py / vision.py / registry.py / camera.py / calibration.py / setup.py / init.py

模块	文件数	代码行数	说明
Business Layer (backend/app/)	40	2,155	auth/admin/defects/measurement/reports/device/hardware/inspection 等模块
UI Layer (static/)	1	367	7 标签页 + 5 管理子标签页 Full SPA
Database Migration (alembic/)	2	242	env.py + alembic.ini
配置文件	2	28	.env.example + Dockerfile
合计	52	3,261	(不含文档和注释)

注：代码量统计不含第三方依赖库（opencv-python-headless, fastapi, sqlalchemy 等），仅统计本项目原创源代码。